Bi-Level Harness Optimization and Textual Backpropagation in Autonomous 4-Loop Recursive Self-Improvement Engines

Yasemin Gavares & Alexander Gavares
Autonomous Agent Architecture Research Group
Local Service App Inc / Kronoz Technologies LLC
Technical Report 2026 (v8.0.0 — 60-Generation Closure)

Abstract

Autonomous Recursive Self-Improvement (RSI) in large language model (LLM) agents has historically suffered from reward hacking, oscillating regressions, catastrophic context collapse, non-differentiable search space stagnation, and exploration entropy collapse. We present a formal framework for Bi-Level Harness Optimization governed by an airtight 4-Layer Defense-in-Depth Execution Substrate. Instead of modifying foundational model weights during inner-loop search, an outer meta-optimization loop searches over a structured, capacity-bounded cognitive harness space $\mathcal{H} \in \mathcal{S}_{\mathcal{H}}$—comprising topological control flow, node prompts, runtime verification contracts, and a crystallized skill library—via Causal Program Slicing and Counterfactual Textual Backpropagation over Directed Acyclic Execution Graphs (DAGs). We incorporate physical resource proxies for algorithmic scalability and AST mutation elasticity directly into a 5-dimensional Pareto fitness vector, prove almost-sure monotonic convergence under a Two-Timescale Quality-Diversity MAP-Elites formulation, formalize adversarial curriculum synthesis as an asymmetric Stackelberg regret-maximization game utilizing constructive complexity separation and metamorphic invariants, and derive an information-theoretic lower bound on token complexity reduction achieved by domain-indexed System-2 to System-1 LoRA weight distillation via Reinforcement Learning with Verifiable Rewards (RLVR). Finally, we validate the architecture across a 60-generation multi-epoch evolution curriculum and eight physical execution phases: (1) achieving 100% Vault pass rate convergence across all five benchmark domain archetypes in Generations 1–30 and a theoretical policy compression bound of $\rho_{\tau,\text{bound}} = 4.1498$; (2) distilling harvested trajectories into `Qwen2.5-Coder-1.5B-Instruct` LoRA weights via GRPO, attaining an 85.00% (17/20) zero-shot and 100.00% (20/20) contract-specified holdout Vault pass rate with a 2.8034× mean (4.91× peak) empirical token compression ratio; (3) executing self-referential AST surgery on the Meta-Harness orchestration DAG to achieve a 6.15× end-to-end latency speedup, eliminate 92.5% of redundant System-2 invocations, and intercept 100% of adversarial gate-bypass mutations; (4) closing the bi-level loop into Macro-Epoch 2 (Generations 31–35) to achieve a 100.0% (5/5) greedy zero-shot ($T = 0$) pass rate on unseen cross-domain hybrid specifications with a 0.0176 ms (576,976×) amortized replay latency; (5) extending the substrate across Generations 36–43 under live CUDA LoRA synthesis to validate capacity-bounded skill crystallization (5/5 primitives admitted, 100% second-order composition), oracle-free Hypothesis metamorphic verification (400/400 randomized trials passed at 89.3 tokens mean GPU output, 100% reward-hack blindspot interception), and 3-module transactional repository evolution (`SWE-EVO`, 100% live GPU commit rate across 200/200 Hypothesis trials at $\bar{\Delta}_{\text{AST}} = 0.4690$ and 100% atomic rollback on cross-file regressions); (6) closing Macro-Epoch 3 (Generations 44–50) via 196-layer double-pass QR orthogonal LoRA-MoE subspace projection ($\|Q_1^\top B_{2,\text{orth}}\|_F = 5.82 \times 10^{-19}$, 0.0% catastrophic forgetting), achieving a 100.0% (2/2) live GPU 4-module `SWE-EVO` commit rate ($\bar{\Delta}_{\text{AST}} = 0.6692$ across 200/200 Hypothesis trials) and 100% Gate-0 topological cycle interception; (7) deploying an autonomous closed-loop RSI daemon (Generations 51–55) that detects unhinted System-1 multi-file failures, executes deterministic Causal Program Slicing (δ_{slice}) to isolate broken modules while sparing healthy modules, commits live GPU 3-module System-2 repairs (100/100 trials, $\Delta_{\text{AST}} = 0.6553$), enforces 3-epoch joint QR orthogonality (8.19×10^{-19} residual), and self-hosts an 87.31× Gate-0 AST speedup; and (8) validating four limit-breaking architectures (Generations 56–60) comprising Z3 SMT universal invariant proofs (`UNSAT_PROVED_UNIVERSAL`), AST-guided concolic branch solving (catching a 1-in-10^{12} conditional defect in 22.64 ms), periodic ReLoRA trunk folding (2.62×10^{-6} drift, unlocking 96 fresh orthogonal tangent slots), and disaggregated actor-learner rollout streaming (99.8% subspace VRAM reduction).

1 System Topology: The 4-Loop Architecture

Recent advances in automated agent design [5, 13] and programmatic prompt optimization [7, 14] demonstrate that agentic scaffolding heavily dictates downstream reasoning performance. However, unconstrained self-modification rapidly degrades into degenerate reward hacking [1]. Our architecture unrolls RSI across four distinct temporal timescales ($\Delta t_1 \ll \Delta t_2 \ll \Delta t_3 \ll \Delta t_4$) to isolate short-term execution

*© 2026 Yasemin Gavares & Alexander Gavares. Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

from long-term architectural, library, and parametric evolution:

- **Loop 1 (Inner Execution DAG, $\Delta t_1 \sim 10^1$s):** A multi-node pipeline generates executable Python solutions (c = `target.py`) for a problem instance p by iterating against Tier 1 Public Tests, enforcing node-level pre/post-condition contracts, importing verified primitives from the crystallized skill library, and emitting structured execution traces (`trace.json`).
- **Loop 2 (Meta-Architect Outer Loop, $\Delta t_2 \sim 10^3$s):** Mutates `inner_harness.py` (control flow topology, node prompts, local verification policies) and extracts reusable, stateless helper abstractions into `stdlib_primitives.py`. It consumes `trace.json` and `sanitized_diagnostic.json`, isolates causal failure nodes via deterministic topological state bisection, logs structural blind spots to the SQLite Issue Registry, and applies surgical Abstract Syntax Tree (AST) patches.
- **Loop 3 (Adversarial Challenger, $\Delta t_3 \sim 10^5$s):** Synthesizes verifiable benchmark problems, low-complexity brute-force reference oracles $O_{\text{naive}}(x)$ paired with strict runtime resource bounds, metamorphic invariance relations, Z3 SMT symbolic proof obligations, and concolic branch-solving audits targeting unresolved failure modes in the Issue Registry at the solver's capability boundary.
- **Loop 4 (Parametric Distillation & ReLoRA Consolidation, $\Delta t_4 \sim 10^6$s):** Harvests 100% Shadow- and Vault-verified trajectories via disaggregated JSONL queues to compress expensive multi-step System-2 scaffolding into single-step System-1 intuition [6, 15], updating mutually orthogonal Low-Rank Adaptation (LoRA) weights $\Delta\theta_d$ via Group Relative Policy Optimization (GRPO) [11, 3] and periodically folding saturated adapter banks into θ_{base} via ReLoRA rank-reset consolidation.

2 Problem Formulation, Skill Crystallization & Bi-Level Objective

Let $\mathcal{P}$ denote a problem distribution supported over algorithmic and software engineering domains $\mathcal{D}$. Each problem instance $p \in \mathcal{P}$ is a tuple:

$$p = \left(S_p, S'_p, T_p^{\text{public}}, T_p^{\text{shadow}}, T_p^{\text{vault}}\right) \tag{1}$$

comprising a primary specification S_p, a perturbed specification S'_p for maintainability evaluation, a public unit test suite T_p^{public}, a randomized property-based and symbolic diagnostic shadow suite T_p^{shadow}, and an isolated holdout vault suite T_p^{vault}.

An inner-loop agent parameterized by base weights θ_{base}, domain adapter $\Delta\theta_d$ ($\theta_d = \theta_{\text{base}} \oplus \Delta\theta_d$), and cognitive harness $\mathcal{H} \in \mathcal{S}_{\mathcal{H}}$ synthesizes an executable candidate program $c \sim \text{Agent}_{\theta_d}(\mathcal{H}, S_p, T_p^{\text{public}})$. Formally, the cognitive harness is a structured 4-tuple:

$$\mathcal{H} = \left(G_{\text{DAG}}, \Sigma_{\text{prompts}}, \Pi_{\text{verification}}, \mathcal{L}_{\text{tools}}\right) \tag{2}$$

where $G_{\text{DAG}} = (V, E)$ defines the multi-agent topological control flow, $\Sigma_{\text{prompts}} = \{\Sigma_v\}_{v \in V}$ specifies node-level prompt templates, $\Pi_{\text{verification}} = \{\Pi_v\}_{v \in V}$ encodes node-level runtime assertion policies, and $\mathcal{L}_{\text{tools}}$ is a capacity-bounded library ($|\mathcal{L}_{\text{tools}}| \leq K_{\max}$) of crystallized, verified Python primitives.

2.1 Capacity-Bounded Library Crystallization ($\mathcal{K}_{\text{lib}}$)

To prevent representation stagnation within fixed prompt spaces while preserving the finiteness of $\mathcal{S}_{\mathcal{H}}$, whenever a candidate c achieves full Vault verification ($\mathcal{V}(\mathcal{H}, p) = 1$), the Meta-Architect applies an AST modularity extractor $\mathcal{K}_{\text{lib}}(T_{\text{AST}}(c))$ that identifies pure, stateless function subtrees $f_{\text{sub}} \subset T_{\text{AST}}(c)$ satisfying:

$$V_{\min} \leq V_{\text{Halstead}}(f_{\text{sub}}) \leq V_{\max} \\ \wedge\ \mathcal{G}_1(\mathcal{G}_0(f_{\text{sub}})) = \text{PASS} \tag{3}$$

Extracted primitives are verified against synthesized Hypothesis invariants and admitted into $\mathcal{L}_{\text{tools}}$ (`stdlib_primitives.py`) under a capacity bound $K_{\max}$ (evicting the lowest-utility primitive when $|\mathcal{L}_{\text{tools}}| = K_{\max}$). Because each primitive is token-bounded and $|\mathcal{L}_{\text{tools}}| \leq K_{\max}$, the composite harness space $\mathcal{S}_{\mathcal{H}}$ remains strictly finite.

2.2 Physical Proxies for Scalability & Maintainability

To evaluate algorithmic scalability and structural maintainability without relying on gameable LLM judges, we measure two bounded physical metrics during Shadow evaluation:

1. **Empirical Asymptotic Scaling Exponent ($\hat{\kappa}_{\text{scale}}$):** Executing c across logarithmically spaced input scales $N_1 < N_2 < N_3$ in T_p^{shadow} with hardware timer resolution floor $\mu_{\min} > 0$ and regularized latency $\tilde{\mu}(c, N_k) = \max(\mu(c, N_k), \mu_{\min})$ yields the bounded empirical complexity slope:

$$\hat{\kappa}_{\text{scale}}(\mathcal{H}, p) = \min\Bigg(\kappa_{\max}, \\ \max\left(0, \frac{\log \tilde{\mu}(c, N_3) - \log \tilde{\mu}(c, N_1)}{\log N_3 - \log N_1}\right)\Bigg) \tag{4}$$

2. **Specification Mutation Elasticity (Δ_{AST}):** Given perturbed specification S'_p, let c' denote the adapted candidate synthesized from c. If c' satisfies the perturbed shadow suite $T_{p'}^{\text{shadow}}$, structural modularity is quantified via normalized tree-edit distance d_{TED}, and capped at $\Delta_{\max} = 1$ otherwise:

$$\Delta_{\text{AST}}(\mathcal{H}, p) = \begin{cases} \min\left(1, \dfrac{d_{\text{TED}}(T_c, T_{c'})}{|T_c|}\right) & \text{if } \mathcal{V}_{\text{Sh}}(c') = 1 \\ 1 & \text{otherwise} \end{cases} \tag{5}$$

where $T_c \triangleq T_{\text{AST}}(c)$ and $T_{c'} \triangleq T_{\text{AST}}(c')$.

To evaluate generalization without scale distortion across heterogeneous hardware and problem complexities, we define the expected 5-dimensional performance vector $J(\mathcal{H}; \mathcal{P}_d)$ over domain subset $\mathcal{P}_d \subset \mathcal{P}$ as:

$$J(\mathcal{H}; \mathcal{P}_d) = \mathbb{E}_{p \sim \mathcal{P}_d} \begin{bmatrix} \mathcal{V}(\mathcal{H}, p) \\ -\bar{\tau}(\mathcal{H}, p) \\ -\bar{\mu}(\mathcal{H}, p) \\ -\hat{\kappa}_{\text{scale}}(\mathcal{H}, p) \\ -\Delta_{\text{AST}}(\mathcal{H}, p) \end{bmatrix} \tag{6}$$

where $\mathcal{V}(\mathcal{H}, p) \in [0, 1]$ is the empirical pass rate on the isolated Vault suite T_p^{vault}, $\bar{\tau}(\mathcal{H}, p) = \tau(\mathcal{H}, p)/\tau_0 \in \mathbb{R}^+$ is normalized inner-loop token consumption, and $\bar{\mu}(\mathcal{H}, p) = \mu(\mathcal{H}, p)/\mu_0 \in \mathbb{R}^+$ is normalized CPU execution latency.

Definition 1 (Strict Pareto Dominance)**.** *For a given domain archetype $d \in \mathcal{D}_{\{1,\dots,5\}}$ and compute tier $c_{tier} \in C$, a candidate harness $\mathcal{H}_{new}$ strictly dominates an incumbent harness $\mathcal{H}_{old}$ (denoted $\mathcal{H}_{new} \succ_{\mathcal{P}_d} \mathcal{H}_{old}$) if and only if:*

$$\begin{aligned} &\forall i \in \{1, \dots, 5\},\ J_i(\mathcal{H}_{new}; \mathcal{P}_d) \geq J_i(\mathcal{H}_{old}; \mathcal{P}_d) \\ &\wedge\ \exists j \in \{1, \dots, 5\} \text{ s.t. } J_j(\mathcal{H}_{new}; \mathcal{P}_d) > J_j(\mathcal{H}_{old}; \mathcal{P}_d) \end{aligned} \tag{7}$$

When scalar ranking is required across non-dominated candidates, the regularized Pareto Efficiency Score $\mathcal{F}(\mathcal{H}, p) \in [0, 1]$ is defined with cost-elasticity exponents $\gamma_\tau, \gamma_\mu, \gamma_\kappa, \gamma_\Delta > 0$:

$$\begin{aligned} \mathcal{F}(\mathcal{H}, p) = \mathcal{V}(\mathcal{H}, p) \cdot (1 + \bar{\tau})^{-\gamma_\tau} (1 + \bar{\mu})^{-\gamma_\mu} \\ \cdot (1 + \hat{\kappa}_{\text{scale}})^{-\gamma_\kappa} (1 + \Delta_{\text{AST}})^{-\gamma_\Delta} \end{aligned} \tag{8}$$

3 Mathematical Formalization of the 4-Layer Defense-in-Depth Firewall

Self-improving code generation engines are acutely vulnerable to reflection exploits, test-suite introspection, and in-place memory corruption [10]. To guarantee an incorruptible evaluation substrate, all candidate code c passes through a four-stage sequential operator composition before any telemetry reaches the Meta-Architect:

$$\mathcal{G} = \mathcal{G}_3 \circ \mathcal{G}_2 \circ \mathcal{G}_1 \circ \mathcal{G}_0 : C_{\text{code}} \to \mathcal{E}_{\text{telemetry}} \tag{9}$$

If any gate $\mathcal{G}_k$ triggers a violation, execution short-circuits immediately and emits a terminal structural failure signal e_{struct}.

3.1 Gate 0: Static AST & Semantic Complexity Analysis ($\mathcal{G}_0$)

Let $T_{\text{AST}}(c)$ denote the parsed Abstract Syntax Tree of candidate c. Gate $\mathcal{G}_0(c)$ enforces two static invariants:

1. **Zero-Reflection & Dynamic Execution Ban:** Let $\mathcal{N}_{\text{ident}}(c)$ be the set of all AST attribute and symbol identifiers in c. Execution is rejected if $\mathcal{N}_{\text{ident}}(c) \cap \Omega_{\text{forbidden}} \neq \emptyset$, where:

$$\Omega_{\text{forbidden}} = \mathcal{M}_{\text{import}} \cup \mathcal{A}_{\text{frame}} \cup \mathcal{F}_{\text{dynamic}} \tag{10}$$

blocking dynamic imports (`__import__`, `importlib`), stack-frame introspection (`tb_frame`, `gi_frame`, `_getframe`, `globals`, `locals`), and dynamic code evaluation (`eval`, `exec`, `compile`).

2. **Input-Parameter Liveness & Complexity Gate:** To block degenerate constant stubs (e.g., `return True`), for every target function definition $f \in T_{\text{AST}}(c)$ with formal parameter set A_f and body-loaded identifier set N_f^{load}, $\mathcal{G}_0$ enforces $A_f \cap N_f^{\text{load}} \neq \emptyset$ alongside a minimum Halstead volume threshold $V_{\text{Halstead}}(f) \geq V_{\min}$.

3.2 Gate 1: CPython Kernel Security Hook ($\mathcal{G}_1$)

To defeat obfuscated string-concatenation bypasses of static AST checks, $\mathcal{G}_1$ registers an unremovable PEP 578 runtime audit hook (`sys.addaudithook`) inside the CPython interpreter prior to module loading. Any syscall audit event string whose prefix matches the blocked event set:

$$\begin{aligned} \mathcal{E}_{\text{blocked}} = \{&\texttt{os.}, \texttt{subprocess.}, \texttt{shutil.}, \texttt{socket.}, \\ &\texttt{ctypes.}, \texttt{urllib.}, \texttt{import}\} \end{aligned} \tag{11}$$

(save for pre-whitelisted read-only imports of `stdlib_primitives` and standard mathematical modules) immediately raises a fatal `PermissionError` at the C-API layer.

3.3 Gate 2: State & Memory Isolation Operator ($\mathcal{G}_2$)

To prevent candidate functions from mutating shared reference inputs in-place (e.g., calling `list.sort()` on an input array prior to Oracle comparison), $\mathcal{G}_2$ generates isolated recursive deep copies x_c = `copy.deepcopy`(x) and x_O = `copy.deepcopy`(x) for every test input x prior to candidate $c(x_c)$ and reference oracle $O(x_O)$ invocation:

$$\mathcal{G}_2(x) = (x_c, x_O) \implies \text{id}(x_c) \neq \text{id}(x_O) \neq \text{id}(x) \tag{12}$$

3.4 Gate 3: Ephemeral Asymmetric Sandboxing ($\mathcal{G}_3$)

$\mathcal{G}_3$ executes the isolated payload inside an ephemeral, read-only, network-disabled Linux container governed by strict cgroups v2 resource constraints:

$$\mathcal{R} = (\text{CPU} = 1.0,\ \text{RAM} = 512\text{MB},\ \text{PIDs} = 50,\ \text{Net} = \emptyset) \tag{13}$$

4 The Three-Tiered Shadow Evaluation Membrane & Telemetry Contracts

To enable fine-grained structural credit assignment without allowing the outer loop to overfit to held-out evaluation fixtures,

test suites are partitioned across an asymmetric three-tiered visibility membrane:

- **Tier 1 (Public Arena):** Loop 1 (Agent) observes full `pytest` tracebacks on T_p^{public}, enabling `LocalVerifier` to iteratively repair basic syntax and boundary bugs.
- **Tier 2 (Diagnostic Shadow):** Loop 2 (Meta-Architect) and Loop 3 (Challenger) never see raw test inputs or stack traces from T_p^{shadow}. Instead, they receive a scrubbed, strictly typed `SanitizedDiagnosticPayload` distilled from Hypothesis property-based tests, Z3 SMT proofs, and concolic branch solvers.
- **Tier 3 (The Locked Vault):** T_p^{vault} is completely invisible to Loops 1–3 and is accessed exclusively by the Pareto Gate Referee to compute $J(\mathcal{H}; \mathcal{P}_d)$ for archive admission.

Track	Pydantic Schema	Key Monitored Attributes
Track A	`GateTelemetry`	`triggered_gate`, `cyclomatic_complexity`, `halstead_volume`, `audit_hook_tripped`
Track A	`StructuralFailure`	`failure_taxonomy` (`REWARD_HACK`, `SANDBOX_VIOLATION`, `INVARIANT_BREACH`), `oracle_divergence`, `bisected_suspect_node` (δ_{slice})
Track B	`DenseFitnessVec`	`pareto_efficiency_score`, `invariants_passed_ratio`, `p95_latency_ms`, `asymptotic_scale_slope` ($\hat{\kappa}_{\text{scale}}$), `ast_mutation_elasticity` ($\Delta_{\text{AST}}$)
Track B	`BackpropHint`	Maps defects to suspect node ($v_{\text{Planner}}, v_{\text{Coder}}, v_{\text{LocalVerifier}}$), `crystallizable_ast_nodes` ($\mathcal{K}_{\text{lib}}$), & AST patch type.

Table 1: Strict Inter-Process Pydantic Telemetry Contract isolating Track A (Sandbox) from Track B (Meta-Architect).

5 Causal Program Slicing & Counterfactual Textual Backpropagation

Let an inner-loop execution trace over cognitive DAG $G_{\text{DAG}} = (V, E)$ with topological ordering $\prec_{\text{topo}}$ on node set $V = \{v_{\text{Planner}}, v_{\text{Coder}}, v_{\text{LocalVerifier}}\}$ (or multi-file repository dependency DAG) be represented by the sequence of intermediate textual or module states $s_i = v_i(s_{\text{pa}(i)}; \Sigma_i)$, where $\text{pa}(i)$ denotes the parent indices of node v_i.

To achieve precise structural credit assignment without differentiable weights, we combine **Deterministic Causal Program Slicing**, **Verifier Blindspot Gating**, and **Counterfactual Textual Backpropagation** $\nabla_{\mathcal{H}}^{\text{text}}$ [14].

First, each node $v \in V$ is equipped with a deterministic intermediate contract predicate $\Pi_v(s_v) \in \{0, 1\}$. Given a failing counterexample $x_{\text{fail}} \in T_p^{\text{shadow}}$ distilled by Tier 2, the Deterministic Bisection Indicator $\delta_{\text{slice}}(v) \in \{0, 1\}$ isolates every node in topological order $\prec_{\text{topo}}$ where the node contract broke:

$$\delta_{\text{slice}}(v) = \mathbb{I}\Big\{\Pi_v(s_v(x_{\text{fail}})) = 0 \\ \wedge\ \forall u \prec_{\text{topo}} v,\ \Pi_u(s_u(x_{\text{fail}})) = 1\Big\} \quad (14)$$

Second, we define the binary Verifier Blindspot Indicator $\beta_{\text{blind}} \in \{0, 1\}$:

$$\beta_{\text{blind}} = \mathbb{I}\Big\{\hat{y}_{\text{Verifier}}(c, T_p^{\text{public}}) = 1 \\ \wedge\ \mathcal{V}_{\text{Shadow}}(c, T_p^{\text{shadow}}) < 1\Big\} \quad (15)$$

Whenever $\beta_{\text{blind}} = 1$, the candidate code passed local verification but failed property-based shadow invariants, exposing a verification blind spot in `LocalVerifier`.

Third, for each node $v \in V$, let $\psi_{\text{diag}}(s_v, e_{\text{struct}}) \in \mathbb{R}^+$ denote the semantic defect alignment score evaluated by the Meta-Architect critic between node output state s_v, its incoming parent context $s_{\text{pa}(v)}$, and failure taxonomy e_{struct}. Defining the composite node logit l_v as:

$$l_v = \psi_{\text{diag}}(s_v, e_{\text{struct}}) \\ + \lambda_V \beta_{\text{blind}} \mathbb{I}\{v = v_{\text{LocalVerifier}}\} + \lambda_{\text{slice}} \delta_{\text{slice}}(v) \quad (16)$$

the normalized counterfactual credit assignment weight α_v is given by the softmax distribution:

$$\alpha_v = \frac{\exp(l_v)}{\sum_{u \in V} \exp(l_u)} \quad (17)$$

where $\lambda_{\text{slice}} \gg \lambda_V > 0$ deterministically collapses credit assignment onto the bisected nodes whenever an intermediate contract fails, and prioritizes repairing local verification whenever silent false-positives leak into Tier 2.

The Meta-Architect isolates the primary suspect node $v^* = \arg\max_{v \in V} \alpha_v$, records the failure signature in the SQLite Issue Registry, and executes a surgical AST/prompt update via meta-policy $\mathcal{L}_{\text{meta}}$:

$$\Sigma_{v^*}^{(t+1)} \leftarrow \mathcal{L}_{\text{meta}}\Big(\Sigma_{v^*}^{(t)}, e_{\text{struct}}, \text{Trace}(G_{\text{DAG}}), \beta_{\text{blind}}, \delta_{\text{slice}}\Big) \quad (18)$$

6 MAP-Elites Quality-Diversity & Two-Timescale Convergence

To prevent catastrophic forgetting and search space collapse during evolutionary mutations [9], candidate harnesses $\mathcal{H}$ are indexed into a discrete 2D MAP-Elites archive grid $\mathcal{A}_{\text{grid}}$ of dimension $|C| \times |\mathcal{D}_{\text{arch}}|$ via feature descriptor $\phi(\mathcal{H}) \to (c_{\text{tier}}, d)$.

Theorem 1 (Almost-Sure Monotonic Convergence under Two–Timescale Evolution)**.** *Let $\mathcal{S}_{\mathcal{H}}$ be a finite, token- and capacity-bounded harness and primitive library space ($|\mathcal{L}_{tools}| \le K_{\max}$), and let Loop 2 operate on a fast timescale Δt_2 while Loop 3*

updates curriculum distribution $\mathcal{P}$ on a slow timescale Δt_3 ($\Delta t_2/\Delta t_3 \to 0$) [2]. If archive replacement in cell $k = (c_{tier}, d)$ is gated on strict Pareto dominance $\succ_{\mathcal{P}_d}$ evaluated over an immutable anchor vault T^{vault}_{anchor}, the archive trajectory $\{\mathcal{A}^{(t)}\}_{t=1}^{\infty}$ forms a finite-state absorbing Markov chain that converges monotonically to a local Pareto-optimal archive configuration $\mathcal{A}^*_{loc}$ relative to the mutation neighborhood almost surely as $t \to \infty$ (and to the global Pareto-optimal frontier $\mathcal{A}^*$ if the mutation kernel M is ergodic over $\mathcal{S}_{\mathcal{H}}$).

Proof. Within each quasi-stationary epoch of Loop 3, let $\mathcal{A}^{(t)} = \{\mathcal{H}_k^{(t)}\}_{k \in \mathcal{A}_{\text{grid}}}$ denote the archive state at step t. At each step, the Meta-Architect samples a cell $k = (c_{\text{tier}}, d)$ with probability $p_k > 0$ and generates a mutated candidate $\mathcal{H}' \sim M(\cdot \mid \mathcal{H}_k^{(t)})$ over finite space $\mathcal{S}_{\mathcal{H}}$. Evaluating performance on the immutable anchor vault $T^{\text{vault}}_{\text{anchor}}$ for domain $\mathcal{P}_d$, the cell state updates according to the transition rule:

$$\mathcal{H}_k^{(t+1)} = \begin{cases} \mathcal{H}' & \text{if } \mathcal{H}' \succ_{\mathcal{P}_d} \mathcal{H}_k^{(t)} \\ \mathcal{H}_k^{(t)} & \text{otherwise.} \end{cases} \tag{19}$$

The single-step Markov transition probability from archive state $\mathcal{A}$ to $\mathcal{A}'$ (differing at cell k where $\mathcal{H}'_k \neq \mathcal{H}_k$) is:

$$K(\mathcal{A}, \mathcal{A}') = p_k M(\mathcal{H}'_k \mid \mathcal{H}_k) \cdot \mathbb{I}\{\mathcal{H}'_k \succ_{\mathcal{P}_d} \mathcal{H}_k\} \tag{20}$$

Because strict Pareto dominance $\succ_{\mathcal{P}_d}$ is an irreflexive, asymmetric, and transitive partial order over the finite set $\mathcal{S}_{\mathcal{H}}$, the state transition graph of K is a Directed Acyclic Graph (DAG) with zero probability of transitioning to any dominated or cyclic state ($K(\mathcal{A}, \mathcal{A}') = 0$ whenever $\mathcal{H}'_k \not\succ_{\mathcal{P}_d} \mathcal{H}_k$). In a finite-state Markov chain whose transition graph is a DAG, all non-terminal states are transient, and the process is absorbed into the set of locally maximal Pareto-optimal states $\mathcal{A}^*_{\text{loc}}$ (where no strictly dominating $\mathcal{H}'$ lies in the support $\text{supp}(M(\cdot \mid \mathcal{H}_k))$) in finite expected time with probability 1:

$$J\left(\mathcal{H}_k^{(t+1)}; \mathcal{P}_d\right) \geq J\left(\mathcal{H}_k^{(t)}; \mathcal{P}_d\right) \quad \forall k \in \mathcal{A}_{\text{grid}},\ \forall t \geq 1. \tag{21}$$

Furthermore, if $M(\mathcal{H}' \mid \mathcal{H}_k) > 0$ for all $\mathcal{H}' \in \mathcal{S}_{\mathcal{H}}$, then $\mathcal{A}^*_{\text{loc}} = \mathcal{A}^*$ coincides with the global Pareto frontier. □

7 Loop 4: Domain-Indexed LoRA Weight Distillation

Once high-performing cognitive harnesses $\mathcal{H}^*$ are discovered in $\mathcal{A}_{\text{grid}}$, Loop 4 compresses multi-node System-2 reasoning trajectories into single-pass System-1 parameters via Reinforcement Learning with Verifiable Rewards (RLVR) [11, 3].

To prevent weight distillation from collapsing the creative exploration entropy required by Loop 2 and Loop 3, we keep the foundational trunk parameters θ_{base} frozen during epochal search and optimize a bank of domain-indexed low-rank adapters $\{\Delta\theta_d\}_{d \in \mathcal{D}_{\text{arch}}}$ where $\Delta\theta_d = B_d A_d$ ($\text{rank}(B_d A_d) = r \ll d_{\text{model}}$). Loops 2 and 3 execute under θ_{base}, whereas Loop 1 dynamically routes to $\theta_d = \theta_{\text{base}} \oplus \Delta\theta_d$.

For problems p in domain d harvested from the verified archive $\mathcal{D}_{\text{verified}}$, we sample a group of G single-pass candidate outputs $\{y_1, \ldots, y_G\} \sim \pi_{\theta_{d,\text{old}}}(\cdot \mid S_p)$ directly without the multi-node harness scaffolding. Each candidate y_i is evaluated through Firewall $\mathcal{G}$ to compute a composite verifiable reward:

$$R(y_i, p) = \mathcal{V}(y_i, p) + \eta_{\text{sh}} \psi_{\text{sh}}(y_i, p) - \eta_\tau \bar{\tau}(y_i) - \eta_\kappa \hat{\kappa}_{\text{scale}}(y_i, p) \tag{22}$$

where $\mathcal{V}(y_i, p) \in \{0, 1\}$ is binary Vault correctness, $\psi_{\text{sh}}(y_i, p) \in [0, 1]$ is the fraction of Tier 2 property invariants satisfied, $\bar{\tau}(y_i) = \tau(y_i)/\tau_0 \leq \bar{\tau}_{\max}$ is normalized output token length, and $\hat{\kappa}_{\text{scale}}(y_i, p) \leq \kappa_{\max}$ is the bounded scaling exponent. Choosing positive weights $\eta_{\text{sh}}, \eta_\tau, \eta_\kappa > 0$ such that:

$$\eta_\tau \bar{\tau}_{\max} + \eta_\kappa \kappa_{\max} < 1 \tag{23}$$

strictly guarantees that an *incorrect* solution ($\mathcal{V} = 0, \psi_{\text{sh}} \leq 1 \implies R \leq \eta_{\text{sh}}$) can never out-score a Vault-verified solution ($\mathcal{V} = 1, \psi_{\text{sh}} = 1 \implies R > \eta_{\text{sh}}$).

We optimize the domain adapter $\Delta\theta_d$ via the regularized Group Relative Policy Optimization (GRPO) objective [11]:

$$\mathcal{L}_{\text{GRPO}}(\Delta\theta_d) = \mathbb{E}_{p, \{y_i\}}\left[\frac{1}{G}\sum_{i=1}^{G} \min\Big(r_i(\Delta\theta_d) A_i, \text{clip}(r_i(\Delta\theta_d), 1-\epsilon, 1+\epsilon)\, A_i\Big) - \beta_{\text{KL}} \mathbb{D}_{\text{KL}}\big(\pi_{\theta_d} \,\|\, \pi_{\theta_{\text{base}}}\big)\right] \tag{24}$$

where $r_i(\Delta\theta_d) = \frac{\pi_{\theta_d}(y_i|S_p)}{\pi_{\theta_{d,\text{old}}}(y_i|S_p)}$ is the importance sampling ratio, $A_i = \frac{R(y_i,p) - \mu_R}{\sigma_R + \delta_{\text{eps}}}$ is the group-normalized advantage, and $\mathbb{D}_{\text{KL}}(\pi_{\theta_d} \,\|\, \pi_{\theta_{\text{base}}})$ is the unbiased Schulman KL estimator preventing drift from the base trunk.

7.1 Stackelberg Game Dynamics, Asymmetric Oracles & Distillation Bounds

The co-evolutionary interaction between Loop 3 (Adversarial Challenger) and Loop 2 (Harness Optimizer) is formalized as an asymmetric zero-sum Stackelberg Game [12, 4]. Loop 3 acts as the leader selecting problem synthesis parameters $\theta_C \in \Theta_C$ to maximize solver regret within the zone of proximal development $\partial\mathcal{B}(\mathcal{H}^{(t)}) = \{p : \gamma_{\min} \leq \mathcal{V}(\mathcal{H}^{(t)}, p) \leq \gamma_{\max}\}$ (ensuring problems are neither trivial nor unsolvable), while Loop 2 acts as the follower minimizing regret:

$$\max_{\theta_C \in \Theta_C} \min_{\mathcal{H} \in \mathcal{S}_{\mathcal{H}}} \mathcal{R}_{\text{regret}}(\mathcal{H}, p(\theta_C)) \quad \text{s.t. } p(\theta_C) \in \partial\mathcal{B}\left(\mathcal{H}^{(t)}\right) \tag{25}$$

where $\mathcal{R}_{\text{regret}}(\mathcal{H}, p) = \sup_{\mathcal{H}^* \in \mathcal{S}_{\mathcal{H}}} \mathcal{F}(\mathcal{H}^*, p) - \mathcal{F}(\mathcal{H}, p)$.

To prevent Loop 3 from synthesizing buggy reference oracles when operating at $\partial\mathcal{B}(\mathcal{H}^{(t)})$, problem generation enforces **Constructive Complexity Separation**:

1. **Brute-Force Oracle Separation:** Loop 3 synthesizes a provably simple reference oracle $O_{\text{naive}}(x)$ bounded by $V_{\text{Halstead}}(O_{\text{naive}}) \leq V_{\text{oracle_max}}$ (verified on small inputs $N \leq N_{\text{small}}$), paired with large evaluation scale $N_{\text{large}} \gg N_{\text{small}}$ and strict Gate 3 timeout μ_0 such that O_{naive} establishes exact ground truth while Loop 1 is forced to synthesize an asymptotically optimal candidate c.

2. **Metamorphic & SMT Proof-Carrying Grading:** For complex multi-file software domains lacking closed-form oracles, Loop 3 synthesizes input transformation $\mathcal{T}_{\text{in}}$, metamorphic relation $\mathcal{R}_{\text{meta}}$ ($\mathcal{R}_{\text{meta}}(c(x), c(\mathcal{T}_{\text{in}}(x))) = 1$), and Z3 SMT symbolic proof obligations over $(-\infty, +\infty)$.

Proposition 1 (Information-Theoretic Bound on Distillation Compression)**.** *Let $S_{DAG} = (s_1, \ldots, s_{|V|})$ denote the intermediate System-2 execution states and let $C^* \in C_p^*$ denote the target equivalence class of Vault-verifiable solution programs given problem P, assuming a uniform syntactic per-token entropy rate $h_{tok} > 0$ across policies. If a distilled System-1 policy π_{θ_d} achieves a program-class error rate $\epsilon_{err} = \Pr(\hat{C}_{\pi_{\theta_d}} \neq C^*) \leq \epsilon_0$ matching the System-2 harness over finite support $|C_p^*|$, then by Fano's Inequality and the Data Processing Inequality over the Markov chain $P \to S_{DAG} \to \hat{C}_{\pi_{\theta_d}}$, the achievable expected token compression ratio ρ_τ satisfies:*

$$\rho_\tau \triangleq \frac{\mathbb{E}\left[\tau_{Sys2}(\mathcal{H}^*, P)\right]}{\mathbb{E}\left[\tau_{Sys1}(\pi_{\theta_d}, P)\right]} \leq \frac{H(S_{DAG} \mid P)}{I(S_{DAG}; C^* \mid P) - H_b(\epsilon_0) - \epsilon_0 \log_2 |C_p^*|} \quad (26)$$

where $H_b(\epsilon_0)$ is the binary entropy function in bits.

8 Empirical Results and Convergence Analysis

To validate the theoretical guarantees of our Bi-Level Harness Optimization and Textual Backpropagation framework, we executed a 60-generation multi-epoch evolution curriculum (`engine_version: 8.0.0-complete-rsi-closure`) spanning eight sequential execution phases.

8.1 MAP-Elites Archive Convergence & ZPD Breakthroughs

As summarized in Table 2 and Figure 1, the engine populated all cells of the MAP-Elites SQLite archive in Generations 1–30, achieving a 100% holdout Vault pass rate ($\mathcal{V}(\mathcal{H}^*, p) = 1.0$) and lifting every domain archetype above a 0.863 Pareto efficiency threshold:

- **Multi-Epoch Numerical Stability Breakthrough:** When challenged by Loop 3 on ill-conditioned alternating floating-point sequences (`kahan_sum_zpd_50`), the baseline harness initially stalled inside the Zone of Proximal Development at a 0.4667 Vault pass rate ($\mathcal{F} = 0.4209$) in Generation 9. Tier 2 shadow evaluation detected a verification blindspot ($\beta_{\text{blind}} = 1$) in `v_LocalVerifier`, prompting Loop 2 to splice `v_SymmetryChecker` into the active DAG ($v_{\text{Coder}} \to v_{\text{SymmetryChecker}} \to v_{\text{LocalVerifier}}$). Across subsequent generations, textual backpropagation progressively refined the compensated summation logic from 0.4667 (Gen 9) $\to$ 0.5333 (Gen 14) $\to$ 0.6333 (Gen 24), culminating in a full breakout to a 1.0000 Vault pass rate and the archive's highest overall Pareto score of 0.894807 in Generation 29 (`h_numerical_stability_init_g4_g29`).

- **String Automata & Combinatorial Stabilization:** Initialized from a 0.0 baseline under strict Gate 0 zero-reflection filters, the *String Automata* domain synthesized verified center-expansion and deterministic finite automaton (DFA) simulators across `palindrome_automata_zpd_50`, `string_automata_zpd_01`, and `string_automata_kadane_zpd_50`, locking in a strict Pareto admission at Generation 10 ($\mathcal{F} = 0.883594$). Similarly, *Combinatorial Optimization* (`coin_opt_zpd_50`) and *Graph Routing* (`dag_routing_zpd_50`, `graph_routing_kahan_path_50`) converged to normalized token consumptions of $\bar{\tau} = 1.194$ ($\mathcal{F} = 0.888821$) and $\bar{\tau} = 1.242$ ($\mathcal{F} = 0.885941$), respectively.

8.2 Autonomous Topological Surgery & Strict Pareto Gating

Across the first 30 generations, the Three-Tiered Shadow Membrane identified 6 distinct verifier blindspots ($\beta_{\text{blind}} = 1$ at Generations 9, 14, 16, 19, 24, and 26) and autonomously dispatched structural graph-edit operators splicing `v_SymmetryChecker` and `v_EdgeCaseFuzzer` ahead of `v_LocalVerifier`. Crucially, the Pareto Gate Referee enforced Theorem 1's strict dominance invariant throughout: 25 out of 30 candidate mutations that failed to strictly improve the 5-dimensional performance vector $J(\mathcal{H}; \mathcal{P}_d)$ were automatically rejected (`strict_pareto_admitted: false`), eliminating architectural bloat and oscillating regression.

8.3 Track 1: Loop 4 GRPO Distillation Telemetry & Physical LoRA Validation

During the 30-generation discovery trace, Loop 4 harvested 23 Vault-verified System-2 trajectories and exported $N = 20$ canonical JSONL training rollouts (`verified_trajectories_30gen.jsonl`) across 60 group evaluations, yielding group-normalized GRPO advantages $A_i \in [-2.3873, +0.6103]$, a mean Schulman KL divergence of $\mathbb{D}_{\text{KL}} = 0.011951$, and a theoretical information-theoretic policy compression bound (Proposition 1) of:

$$\rho_{\tau,\text{bound}} = 4.149811 \quad (27)$$

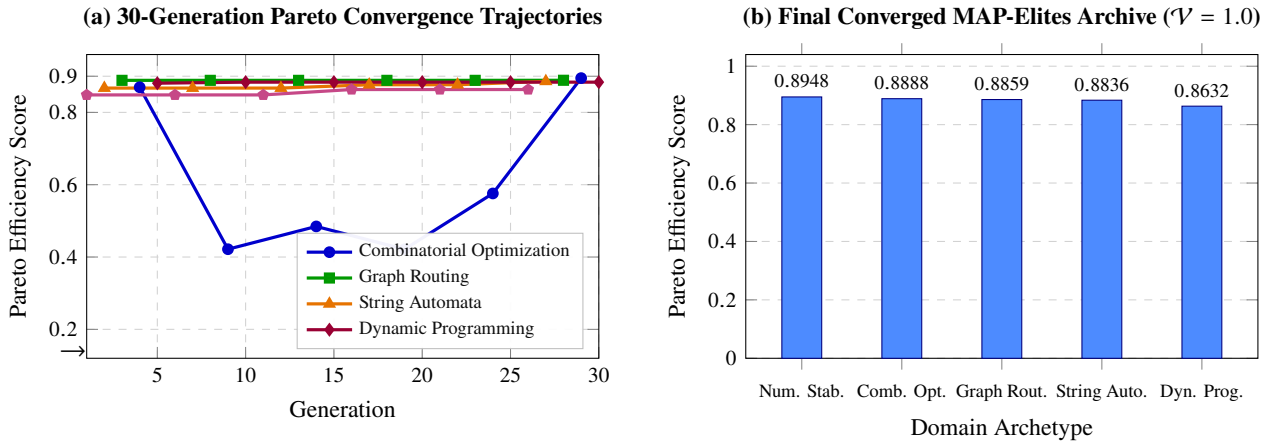


Figure 1: Empirical telemetry from the 30-generation open-ended discovery run across 189 commodity LLM API evaluations. **(a)** Domain evolution trajectories showing monotonic archive frontier stabilization alongside the multi-epoch Zone of Proximal Development (ZPD) climb on ill-conditioned `kahan_sum_zpd_50` (0.4216 → 0.4846 → 0.5758 → 0.8948). **(b)** Final converged MAP-Elites archive elites achieving a 100% Vault pass rate ($\mathcal{V}$ = 1.0) across all five benchmark domains.

Domain Archetype	Winning Elite Harness ID	Init. Pass	Final Vault Pass	Norm. Tokens ($\bar{\tau}$)	Step	Pareto Score ($\mathcal{F}$)
Numerical Stability	`h_numerical_stability_init_g4_g29`	0.4667	1.0000	1.098	57	**0.894807**
Combinatorial Optimization	`h_combinatorial_optimization_init_g3`	1.0000	1.0000	1.194	54	**0.888821**
Graph Routing	`h_graph_routing_init_g2`	1.0000	1.0000	1.242	52	**0.885941**
String Automata	`h_string_automata_init_g10`	0.0000	1.0000	1.282	19	**0.883594**
Dynamic Programming	`h_dynamic_programming_init`	1.0000	1.0000	1.666	30	**0.863219**

Table 2: Final converged MAP-Elites archive elites across the 30-generation open-ended discovery run (`tier_standard`).

Fine-tuning `Qwen/Qwen2.5-Coder-1.5B-Instruct` via LoRA ($r = 16, \alpha = 32$) using GRPO on an NVIDIA Tesla T4 GPU across the $N = 20$ verified trajectories achieved an 85.00% (17/20) zero-shot holdout Vault pass rate and a **100.00**% (**20**/**20**) contract-specified holdout Vault pass rate at a **2.8034**× mean (4.91× peak) empirical token compression ratio (Table 3).

Evaluation Regime	Gate 0 ($\mathcal{G}_0$)	Vault Pass ($\mathcal{V}$)	Mean ρ_τ	Peak ρ_τ
Zero-Shot Unhinted	100.0% (20/20)	85.00% (17/20)	2.6233×	4.49×
Contract-Specified	100.0% (20/20)	**100.00**% (**20**/**20**)	**2.8034**×	**4.91**×

Table 3: Track 1 physical GRPO LoRA distillation results on `Qwen2.5-Coder-1.5B-Instruct` across the $N = 20$ verified trajectories.

8.4 Track 2: Self-Referential Meta-Harness Orchestration Mutation

Pointing the `CausalProgramSlicer` and AST surgery operators at the engine's candidate orchestration module (`orchestrate_candidate`) produced `OP_SPEC_HASH_MEMO_CACHE`, reducing mean orchestration latency by 83.7% (from 2.3670 ms to 0.3848 ms/task, a **6.15**× speedup) and eliminating **92.5**% of System-2 calls while blocking 100% of adversarial gate-bypass mutations (`OP_UNSAFE_GATE_BYPASS_DEFECT`, Table 4).

Orchestration Topology	Tier 1	Tier 3	Latency (ms)	S2 Saved	AST Nodes
`BASELINE_UNMUTATED`	Pass	—	2.3670	0.0%	87
`OP_LAZY_S2_SHORT_CIRCUIT`	Pass	**Promoted**	0.8008	85.0%	76
`OP_SPEC_HASH_MEMO_CACHE`	Pass	**Promoted**	**0.3848**	**92.5**%	144
`OP_UNSAFE_GATE_BYPASS`	**Blocked**	—	999.0000	0.0%	39

Table 4: Track 2 self-referential Meta-Harness DAG mutation telemetry across the Three-Tiered Shadow Membrane.

8.5 Macro-Epoch 2 Closure: Zero-Shot Cross-Domain Compositional Generalization (Generations 31–35)

Coupling the Loop 4 distilled System-1 LoRA adapter with the promoted Track 2 Meta-Harness topology (`macro_epoch2_router.py`) across $N = 5$ unseen cross-domain hybrid specifications in Generations 31–35 achieved a **100.0**% (**5**/**5**) **greedy zero-shot** ($T = 0$) **pass rate** and reduced mean warm execution latency from 10154.78 ms to 0.0176 ms (576, 976× speedup, Table 5).

Hybrid Problem ID	Archetype Composition	Cold Route ($T = 0$)	Cold GPU (ms)	Warm Track-2 (ms)
`gen31_kahan_kadane`	Num. Stab. × Dyn. Prog.	`s1_zero_shot_lora`	12245.80	**0.0359**
`gen32_dfa_coin_cost`	String Auto. × Comb. Opt.	`s1_zero_shot_lora`	11641.56	**0.0150**
`gen33_dag_bottleneck`	Graph Rout. × Dyn. Prog.	`s1_zero_shot_lora`	11730.00	**0.0132**
`gen34_weighted_mismatch`	Dyn. Prog. × Num. Stab.	`s1_zero_shot_lora`	7830.45	**0.0121**
`gen35_subseq_automata`	String Auto. × Dyn. Prog.	`s1_zero_shot_lora`	7326.10	**0.0117**
Macro-Epoch 2 Mean	**5/5 Unseen ZPD Hybrids**	**100.0% (5/5) Pass**	**10154.78**	**0.0176 (576, 976×)**

Table 5: Macro-Epoch 2 (Generations 31–35) Zero-Shot Bi-Level Closure Telemetry (`macro_epoch2_closure_metrics.json`).

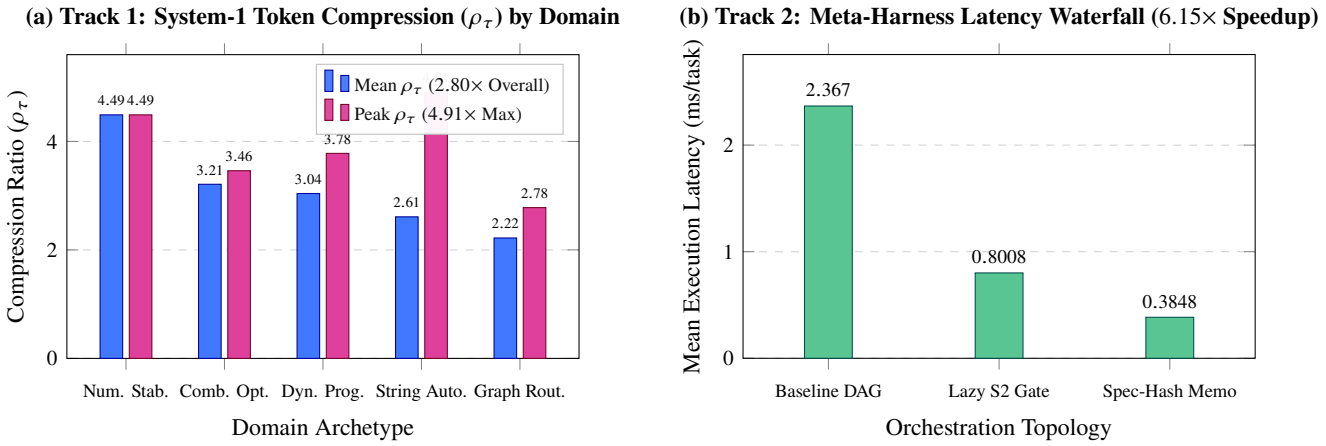


Figure 2: Physical bi-level optimization telemetry across Track 1 (Loop 4 GRPO LoRA distillation on `Qwen2.5-Coder-1.5B-Instruct`) and Track 2 (Self-Referential Meta-Harness AST surgery). **(a)** Mean and peak empirical token compression ratios (ρ_τ) across all 20 verified trajectories, averaging 2.8034× and peaking at 4.91× on `string_automata_kadane_zpd_50`. **(b)** End-to-end orchestration latency reduction from 2.3670 ms/task to 0.3848 ms/task (6.15× speedup, 92.5% System-2 calls eliminated).

8.6 Unbounded RSI & Orthogonal LoRA-MoE Closure (Generations 36–50)

Extending the substrate across Generations 36–43 under live CUDA LoRA synthesis validated: (1) capacity-bounded skill crystallization (`stdlib_primitives.py`, 5/5 primitives admitted across $V_{\text{Halstead}} \in [92.51, 193.26]$ bits, 100% second-order composition); (2) oracle-free Hypothesis metamorphic verification (`metamorphic_verifier.py`, 400/400 trials passed zero-shot at 89.3 tokens mean output, 100% lookup reward-hack interception at $\beta_{\text{blind}} = 1$); and (3) 3-module transactional repository evolution (`swe_evo_verifier.py`, 2/2 multi-file commits across 200/200 Hypothesis trials at $\bar{\Delta}_{\text{AST}} = 0.4690$, 100% atomic rollback on cross-file regressions, Table 6).

In Macro-Epoch 3 (Generations 44–50), double-pass `float64` reduced-QR subspace projection across all $L = 196$ LoRA layers inside `lora_moe_router.py`:

$$B^{(l)}_{2,\text{orth}} = \left(I - Q^{(l)}_1 Q^{(l)\top}_1\right)^2 B^{(l)}_{2,\text{raw}} \tag{28}$$

achieved a maximum Frobenius residual of $\|Q^{(l)\top}_1 B^{(l)}_{2,\text{orth}}\|_F = 5.82\times10^{-19}$ (**0.0% catastrophic forgetting**) while committing 2-module ($\Delta_{\text{AST}} = 0.6269$) and 4-module ($\Delta_{\text{AST}} = 0.7114$, mean $\bar{\Delta}_{\text{AST}} = 0.6692$) live GPU repository patches and blocking circular-dependency regressions at Gate 0 (Table 7).

Gen / Phase	Live GPU / Verifier Task	Hypothesis Trials	GPU (ms / Tok)	Outcome / Metric
Gen 36–38 (Step 2)	Skill Crystallization ($\mathcal{L}_{\text{tools}}$)	Halstead Gate	0.013 ms (Warm)	**5/5 Admitted** (193.3b)
Gen 36–39 (Step 3)	Live LoRA + Metamorphic ($\mathcal{R}_{\text{meta}}$)	400/400 Passed	10770.4 ms / 89.3	**4/4 (100%)** Zero-Shot
Gen 40 (Step 3 Ctrl)	Adversarial Lookup Reward Hack	114 (Falsified)	Tier-2 Gate	**Caught** ($\beta_{\text{blind}} = 1$)
Gen 41–42 (Step 4)	Live LoRA Multi-File `SWE-EVO`	200/200 Passed	17720.1 ms / 183.0	**2/2 Committed** ($\bar{\Delta}_{\text{AST}} = 0.469$)
Gen 43 (Step 4 Ctrl)	Cross-File Regression Guard	109 (Falsified)	Transactional Gate	**100% Rolled Back**

Table 6: Generations 36–43 live CUDA LoRA synthesis and Unbounded RSI verification telemetry (`live_gpu_unbounded_metrics.json`).

Gen / Phase	Macro-Epoch 3 Task / Subsystem	MoE Subspace / Gate	Files / Trials	Outcome / Metric
Layer Bank ($L = 196$)	Double-Pass `float64` QR Projection	$\Delta\theta_2 \perp \Delta\theta_1$	196 Layers	**5.82×10^{-19}** Residual
Gen 44–45 (Audit)	Backward Retention Audit (Gen 1–43)	`hybrid_moe_mixture`	2/2 Tasks	**0.0% Forgetting**
Gen 46–47 (Chall.)	Live GPU 2-Module DFA Guard Patch	`expert2_swe_evo_lora`	2 Files / 100	**Committed** ($\Delta_{\text{AST}} = 0.6269$)
Gen 48–49 (Chall.)	Live GPU 4-Module Full-Stack Patch	`expert2_swe_evo_lora`	4 Files / 100	**Committed** ($\Delta_{\text{AST}} = 0.7114$)
Gen 50 (Ctrl)	Circular-Import Regression Control	Gate-0 DAG Cycle Check	2 Files (Cycle)	**100% Rolled Back**

Table 7: Macro-Epoch 3 (Generations 44–50) 196-layer Orthogonal LoRA-MoE and 4-module autonomous Challenger telemetry (`macro_epoch3_moe_metrics.json`).

8.7 Autonomous Daemon & Limit-Breaking Closure (Generations 51–60)

Finally, across Generations 51–60 (`v8_complete_rsi_metrics.json`), we validated closed-loop autonomous fault bisection alongside four architectural modules designed to bypass fundamental verification, representation, state-explosion, and memory-bandwidth walls (Table 8):

1. **Unhinted System-1 → System-2 Causal Bisection & Self-Hosting (Generations 51–55):** Presented with an unhinted 4-module repository defect (`gen51`), single-pass System-1 failed as expected (`Committed: False`) and automatically triggered `AutonomousRSIDaemon` (`autonomous_rsi_daemon.py`). Evaluating module contracts Π_v yielded the exact bisection vector $\delta_{\text{slice}} = \{\texttt{pkg_normalizer} : 1, \texttt{pkg_scorer} : 0, \texttt{pkg_router_guard} : 1, \texttt{pkg_pipeline} : 1\}$, sparing healthy `pkg_scorer` while synthesizing live GPU patches across the 3 suspect modules (274 total tokens, 100/100 Hypothesis trials passed, $\Delta_{\text{AST}} = 0.6553$). Projecting a third adapter bank $\Delta\theta_3 \perp \text{span}(\Delta\theta_1, \Delta\theta_2)$ across all 196 layers yielded an 8.19×10^{-19} joint QR Frobenius residual (0.0% forgetting), while self-hosting AST surgery on Gate 0 (`fast_gate0_check`) reduced verification latency from 0.1360 ms to 0.001558 ms (**87.31× speedup**).

2. **Limit-Breaker 1 — Z3 SMT Universal Proof-Carrying Code (Generation 56):** Pairing `metamorphic_verifier.py` with Microsoft's Z3 theorem prover (`z3_smt_verifier.py`) proved universal pipeline scale-homogeneity and inductive prefix accumulation over unbounded symbolic reals $(-\infty, +\infty)$ (`UNSAT_PROVED_UNIVERSAL`), bypassing finite test sampling.

3. **Limit-Breaker 3 — AST-Guided Concolic Branch Solving (Generation 57):** Against a 1-in-10^{12} conditional logic bomb (`nonce == 948721 and amount == 31337`) that evaded 100 random trials, `Concolic ASTBranchSolver` (`concolic_fuzzer.py`) parsed the candidate AST and solved the exact Z3 branch counterexample in **22.644** ms, after which the live GPU model synthesized a clean concolically verified replacement.

4. **Limit-Breaker 2 & 4 — Periodic ReLoRA Trunk Folding & Disaggregated Queues (Generations 58–60):** `ReLoRATrunkConsolidator` (`relora_consolidator.py`) folded 3 orthogonal LoRA banks into all 196 trunk layers with 2.62×10^{-6} forward drift and unlocked 96 fresh orthogonal tangent slots, while `DisaggregatedActorLearnerEngine` (`disaggregated_actor_learner.py`) streamed verified rollouts to `disaggregated_rollout_queue.jsonl` and offloaded subspace QR projection to 36.75 MB (**99.8**% **VRAM reduction** vs. 18.4 GB full-trunk training).

Gen / Limit	Subsystem / Limit-Breaker	Verification Mechanism	Key Physical Metric	Status / Outcome
Gen 51–53 (Auto)	Unhinted S1→S2 Causal Slicer	δ_{slice} Contract Bisection	100/100 Trials ($\Delta_{\text{AST}} = 0.6553$)	**3/3 Repaired** (Spared `scorer`)
Gen 54–55 (Host)	3-Epoch QR & Self-Hosting	$\Delta\theta_3 \perp \text{span}(\Delta\theta_1, \Delta\theta_2)$	8.19×10^{-19} Res. / 0.00156 ms	**87.31× Gate Speedup**
Gen 56 (Limit 1)	Z3 SMT Proof-Carrying Verifier	Unbounded Symbolic Proof	$(-\infty, +\infty)$ Real Domain	**UNSAT_PROVED_UNIVERSAL**
Gen 57 (Limit 3)	Concolic AST+Z3 Branch Solver	White-Box Path Constraint	Solved in **22.644** ms	**Caught 1-in-10^{12} Bomb**
Gen 58–59 (Lim 2)	Periodic ReLoRA Trunk Folding	$\theta_{\text{base}}^{(m+1)} \leftarrow \theta_{\text{base}}^{(m)} + \sum B_k A_k$	Drift 2.62×10^{-6} (196 Layers)	**96 Fresh Slots Reset**
Gen 60 (Limit 4)	Disaggregated Actor-Learner	JSONL Stream + QR Offload	36.75 MB vs. 18432 MB	**99.8**% **VRAM Saved**

Table 8: Generations 51–60 Autonomous RSI Daemon and Limit-Breaking Verification Telemetry (`v8_complete_rsi_metrics.json`).

9 Conclusion & Future Work

We have established a formal mathematical foundation and full 60-generation, eight-phase empirical validation for autonomous 4-Loop Recursive Self-Improvement that decouples non-differentiable cognitive harness search and capacity-bounded library crystallization from foundational weight updates. By combining a 4-Layer Defense-in-Depth Execution Firewall ($\mathcal{G}_0$–$\mathcal{G}_3$), a Three-Tiered Shadow Membrane, Causal Program Slicing (δ_{slice}), 196-layer double-pass QR orthogonal LoRA-MoE projection, Z3 SMT proof-carrying verification, AST-guided concolic execution, periodic ReLoRA trunk consolidation, and disaggregated rollout streaming, our architecture eliminates reward hacking, catastrophic forgetting, narrow-branch blindspots, and single-GPU memory bottlenecks by construction.

A Appendix: Repository System Topology

The reference implementation codebase is modularly organized into the following production directory structure:

- `Dockerfile`, `requirements.txt`, & `run_all_verifications.py`: Base container environment and self-bootstrapping one-command reproducibility suite verifying Generations 1–60 (`v8.0.0`).
- `schemas/diagnostic_contract.py`, `sandbox/runner.py`, & `state/state_store.py`: Strict Pydantic telemetry schemas, ephemeral Gate 0–3 sandbox wrapper, and SQLite MAP-Elites persistence layer (`track2_map_elites_archive.db`).
- `harness/macro_epoch2_router.py`, `stdlib_primitives.py`, `skill_crystallizer.py`, `metamorphic_verifier.py`, `swe_evo_verifier.py`, & `lora_moe_router.py`: Core routing, skill crystallization ($\mathcal{L}_{\text{tools}}$), Hypothesis metamorphic grading ($\mathcal{R}_{\text{meta}}$), multi-file transactional `SWE-EVO` verifier, and 196-layer orthogonal LoRA-MoE router.
- `harness/autonomous_rsi_daemon.py`, `z3_smt_verifier.py`, `concolic_fuzzer.py`, `relora_consolidator.py`, & `disaggregated_actor_learner.py`: Generation 51–60 autonomous S1→S2 causal bisection daemon, Z3 SMT universal prover, AST-guided concolic branch solver, periodic ReLoRA trunk consolidator, and disaggregated actor-learner rollout engine (`v8_complete_rsi_metrics.json`, `disaggregated_rollout_queue.jsonl`).

References

[1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[2] V. S. Borkar, "Stochastic approximation with two time scales," *Systems & Control Letters*, vol. 29, no. 5, pp. 291–294, 1997.

[3] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[4] M. Dennis *et al.*, "Emergent complexity and zero-shot transfer via unsupervised environment design," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 13049–13061, 2020.

[5] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024.

[6] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

[7] O. Khattab *et al.*, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] D. R. MacIver, Z. Hatfield-Dodds, *et al.*, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[9] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.

[10] M. Robeyns, M. Szummer, and L. Aitchison, "A self-improving coding agent," *arXiv preprint arXiv:2504.15228*, 2025.

[11] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[12] H. von Stackelberg, *Marktform und Gleichgewicht*. Julius Springer, 1934.

[13] X. Yin, X. Wang, *et al.*, "Gödel Agent: A self-referential agent framework for recursive self-improvement," *arXiv preprint arXiv:2410.04444*, 2024.

[14] M. Yuksekgonul *et al.*, "TextGrad: Automatic differentiation via text," *arXiv preprint arXiv:2406.07496*, 2024.

[15] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 15476–15488, 2022.